

# Design of an Official Document Multi Machine Retrieval System based on NLP

1st Shiqing Cheng\*  
Unit 31511  
Beijing, China  
1940898350@qq.com

2nd Siyu Wang  
Unit 31511  
Beijing, China  
1434717692@qq.com

3rd Renping Chen  
Unit 31511  
Beijing, China  
crp678@qq.com

4th Congliang Luo  
National University of Defense Technology  
Changsha, China  
724261350@qq.com

**Abstract**—Aiming at the problems of insufficient file name retrieval information, large internal retrieval overhead, slow speed and low sharing rate of documents in the existing stand-alone document retrieval software, this paper uses NLP (natural language processing) technology to provide a new system design method for multi-machine retrieval and accurate query of office documents. This method is realized by uploading documents, document text analysis and processing, search engine construction, similar document recommendation, result preview and verification, retrieval result output and so on. This paper will elaborate from five main aspects: the overall system flow, database design, text analysis and processing module design, text retrieval module design and client design.

**Keywords:** NLP; document; multi-machine retrieval; system design

## I. FOREWORD

In the daily office, a large number of official documents will be generated. These documents, as the accumulation of business data and information, often face the situation of query and reading. The existing stand-alone document retrieval software mainly has the following current situation and problems: First, relying on the file name query is limited by the character length, the amount of information that can be retrieved is limited, and the internal text information of the document cannot be queried; The second is to query the internal information of the document through in-depth retrieval. Although the accuracy is high, it needs to analyze and extract all the document text of the system in real time, which has high system overhead and slow speed; Third, the internal document management and retrieval of the Department business system also needs a public “resource pool” to solve the problems of document sharing and online retrieval among multiple users [1].

The system design of this paper adopts NLP technology, through the way of foreground upload, background analysis and processing and terminal reading results[2-3], which overcomes the problems of single machine software retrieving file names, less matching information, inaccurate query results, in-depth analysis and retrieval of internal information of documents, and facing long time-consuming and large system overhead. After the system is put into operation, the front-end user defines the permissions and uploads the documents to the background for analysis and processing, which reduces the retrieval cost of the terminal computer, and realizes the rapid retrieval of documents by building a search engine; Build a

document sharing database to realize document online retrieval; Build a document preview comparison to help users confirm the search results, use the similarity calculation between documents to recommend possible results for users, so that a unit or department can better upload, share and manage a wide range of office documents at each terminal, and provide one click quick query and reading services for each terminal user.

## II. RELATED TECHNOLOGY

### A. TF-IDF algorithm

TF-IDF algorithm is a word frequency statistical algorithm, which is defined as follows:  $TF \times IDF$ , TF is the word frequency, and IDF is the inverse text frequency. The TF calculation formula is:

$$TF = \frac{N(w_i, d)}{S} \quad (1)$$

In formula(1),  $N(w_i, d)$  is the total number of occurrences of a particle  $w_i$  in a document  $d$ , and  $S$  is the total number of particles in the document. TF calculation formula is:

$$IDF = \log\left(\frac{N}{N(w_i)}\right) \quad (2)$$

In formula(2),  $N$  is the total number of all documents in the text processing set, and  $N(w_i)$  is the total number of documents which contain word segmentation  $w_i$  in the text processing set[4-5].

### B. Text similarity calculation

After vectorization of text features, the similarity between texts can be calculated. The similarity calculation is based on VSM (vector space model), that is, after vectorizing the text without considering the correlation between document word segmentation, the relationship between documents is transformed into the distance relationship between two vectors. In this paper, Euclidean distance calculation is used. The specific methods are as follows[6-7]:

Hypothesis: The feature vectors of text 1 is  $(d_1, d_2, \dots, d_n)$ . The feature vectors of text 2 is  $(c_1, c_2, \dots, c_n)$ . Then the similarity between two documents can be calculated according to formula (3):

$$S = \sqrt{(c_1 - d_1)^2 + (c_2 - d_2)^2 + \dots + (c_n - d_n)^2} \quad (3)$$

The smaller the distance between two text vectors, the higher the similarity between the two texts. On the contrary, the larger the distance between two text vectors, the lower the similarity between the texts[8-9].

### C. Whoosh

Whoosh is a text search engine developed by MattChaput using pure python. It was first used in the Houdini help program by SideEffectsSoftware company. Whoosh adopts modular design, takes the field as the index, and provides a fast calling interface, which is very suitable for embedding into Python programs. The construction process of whoosh search engine is as follows: first extract the text from the document, build the retrieval resource pool, then build the index file according to the text information in the resource pool, then sort the retrieval score according to the retrieval keywords provided by the user, and finally output the retrieval results. Since whoosh does not support Chinese word segmentation, word segmentation components are needed to segment words during index construction[10-11].

## III. DESIGN OF OVERALL SYSTEM FLOW

The system of this paper adopts a browser / server architecture and uses Python 3.7.4 language, django3.0 framework. See Figure 1 for the specific overall method flow chart.

Step101: each end user uploads office documents and can set document use permissions as needed. The use permissions are divided into full sharing, specified range sharing or only for this user. The front end of the system is equipped with a specific file type shielding function, so that users can only upload the document types that the system can process. The document types that the system can process are limited to DOC, DOCX, WPS, TXT, XLSX, PDF and other document types that can be parsed by the existing Python toolkit.

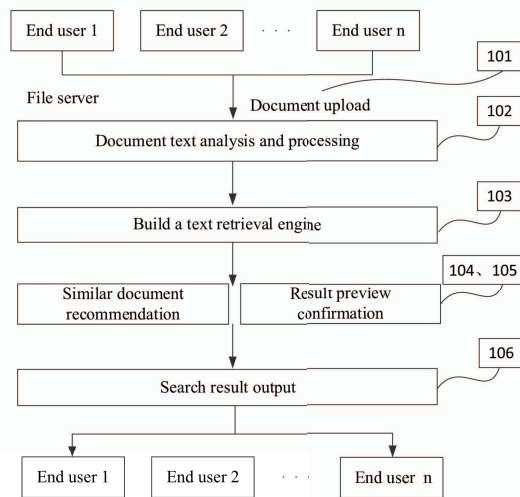


Figure 1. Overall process system.

Step102: document analysis and processing, including reading document text information, extracting document key information, full-text word segmentation, similarity calculation, generating preview file and other steps, and storing the analysis and processing results in the database to prepare for the next full-text retrieval. In this step, through the background server, the document is analyzed and processed in advance and the results are stored in the database. Subsequent users retrieve and preview at the front end, which is called directly, which improves the retrieval speed[12-13].

Step103: build a text retrieval engine, which mainly includes the steps of configuring the search framework, word segmentation of search sentences, improving whoosh search strategy and scoring strategy, visual display of search results, calculation and backward search relative probability, and finally provide users with a fast and efficient retrieval tool.

Step 104 and 105: similar document recommendation and result preview confirmation, both of which are retrieval auxiliary search modules. Similar document recommendation is to count the word segmentation frequency of each document, calculate the similarity with other documents respectively, and recommend according to the calculation results to assist users in finding the results; The result preview confirms the retrieval of the document preview page to provide confirmation for the user's query results

Step106: provide download, print and other export functions for the search results selected by the user.

## IV. DATABASE DESIGN

### A. Relational database

It mainly includes the following information tables:

- Doc\_base\_table. fields include: Doc\_ID, Doc\_Name, Doc\_Aut, Doc\_Sto\_addr, Doc\_View\_addr, etc;
- User\_table. fields include: User\_ID, User\_Name, etc;
- User\_aut\_table. fields include: Doc\_ID, User\_ID, Is\_Permit, etc
- Doc\_detail table. fields include: ID, Doc\_ID, Full\_Text, Title, First\_Title, Sec\_Title, Third\_Title, Num, Secret\_Level, Urgency, etc.
- Doc\_sim\_table. fields include: Doc\_ID, Simi\_Doc\_ID, Simi\_Value, etc

The main relationships between the above information tables are as follows:

Take the user\_aut\_table as the intermediate table between the user\_table and the doc\_base table, so that they can establish a many to many relationship, so that each document can be fully shared, shared within a specified range or used only by the user.doc\_detail\_table and doc\_base\_table is associated with the Doc\_ID field to establish a one-to-one relationship, as shown in Figure 2.

### B. File database

File database includes original document library and preview file library. The original document library stores the original documents uploaded by users, and the preview file library mainly stores the preview files generated by system conversion. The document path is stored in the

relational database, and the document itself is stored in the file database. By separating the document path from the document, the problem of too large document conversion into binary data and slow reading and writing speed in the relational database can be effectively avoided.

## V. DESIGN OF TEXT ANALYSIS AND PROCESSING MODULE

After the user uploads the document, the system automatically reads the basic information of the file from

the original document library and stores it in the document basic information table (doc\_base\_table). The text analysis and processing module starts with the document basic information table, calls the document storage address (doc\_sto\_addr) for processing, and stores the result data in other data tables for subsequent direct call.

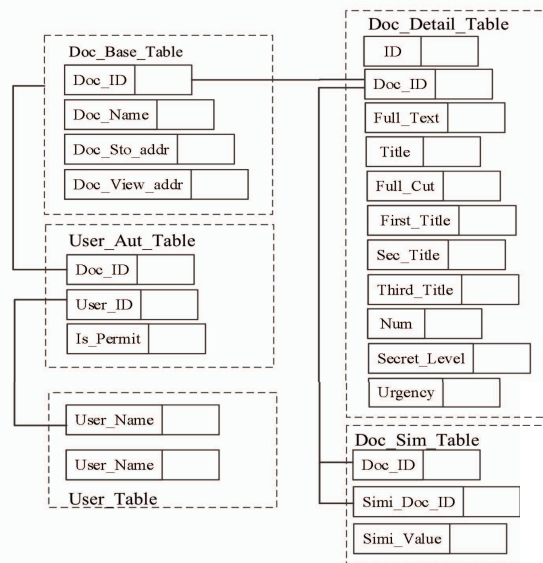


Figure 2. Database main table structure diagram

### A. Read document text information

Call the storage address, use the python toolkit to read the text information in the paragraph paragraph paragraph by paragraph, and read the text information in the table cell by cell. Define the text content read each time as P, and then connect each P with a separator to form a full-text text information, which is stored in the document detail table doc\_detail\_table is in the full\_text field.

### B. Extract document key information

According to the current official document format standard, the title, primary title, secondary title, tertiary title, copy number, confidentiality level, confidentiality period, urgency and other key information contents of each P are matched and extracted, and stored in the field of the doc\_detail\_table. Matching extraction is carried out in two ways: comprehensive model extraction and rule matching. Model extraction mainly uses statistical algorithm, neural network, deep learning and other models[14-15], which is completed through three steps: document key information annotation, model training and input text extraction; Rule matching is mainly composed of font style matching and fixed character matching. Font style matching is mainly judged by the font size, font name, font color, bold and other styles of each P. Fixed character matching mainly uses regular expressions to match fixed words, symbols, punctuation, numbers, etc. in key information.

### C. Full text word segmentation

Use Chinese word segmentation tools such as Jieba, AINLP, HanLP and PkuSeg to segment words in the full\_text field of each doc\_detail\_table. Taking Jieba word segmentation search engine model as an example, the full of an office document full\_text = "provincial Party committee leaders in Zhejiang Ningbo investigation". The output result of word segmentation is: "provincial Party committee / leaders / in / Zhejiang / Ningbo / investigation". For professional and domain office documents, the professional vocabulary in this field is sorted into a professional thesaurus and added to the word segmentation tool thesaurus, so as to improve the accuracy of word segmentation. For example, full\_text = "sensitive element refers to the part of the sensor that can directly feel the measured part". The default word segmentation result is: "sensitive / element / finger / sensor / medium / energy / direct / feel / measured / part". Among them, "sensitive element" is a professional vocabulary. Jieba divided it into "sensitive" and "element" by word segmentation error. After adding "sensitive element" to the word segmentation library, the result of word segmentation again is: "sensitive element / finger / sensor / medium / energy / direct / feeling / measured / part". After word segmentation, it is stored in the Full\_Cut field of the doc\_detail\_table. Full text word segmentation prepares for the next step of counting word frequency, constructing text

features and calculating the similarity between documents[16-18].

#### D. Similarity calculation

From the doc\_detail\_table read the full-text word segmentation of each document in the field of table (full\_cut), create the word bag data structure [19], count the word frequency of each word using TF-IDF algorithm, turn the word frequency of each word into a feature of the document, and then convert the word segmentation results of all documents into a word frequency matrix, and obtain the similar value of the two documents by calculating the Euclidean distance between each two lines of the matrix[20]. Finally, read the similar values of each document and other documents by line and store them in doc\_Sim\_Table. By calculating the similarity between documents in advance in the background of the server, so that the front end can be called directly when it needs to be used without temporary calculation, which saves the calculation waiting time and greatly improves the response speed of the terminal page.

#### E. Generate preview file

Use the python document format conversion toolkit to read the document in the original document library of the file database, then generate the preview file in HTML format by "saving" and store it in the preview file library of the file database. At the same time, store the address of the preview file in the document basic information table, specifically stored in the doc\_view\_addr field of this table. After the preview file is generated, it can be called and embedded by the front-end page to realize the user's page preview of the document. This not only solves the problem that the document editing software cannot be embedded into the front-end page and open the document for preview, but also provides end users with rapid text acquisition and document retrieval result comparison and confirmation services.

### VI. DESIGN OF TEXT RETRIEVAL MODULE

The text search engine is built based on the Django open source search framework Haystack, and the search engine uses Whoosh2.7 written by pure Python language. Based on the framework and engine, the system is redesigned and improved, and the visual display of the retrieval result data is designed[21-24].

#### A. Configure search framework

Use Full\_Text, Title, First\_Title, Sec\_Title, Third\_Title, Num and secret\_Level, urgency and other key information fields of the doc\_detail\_table to build indexes, and from all of the text choose the index which contains the Full\_Text field set as the main index, and other title and key information fields are used to adjust the scoring strategy. In view of the fact that users often upload shared documents in their actual work, set the index to the automatic update mode, and automatically regenerate the index when users add, modify and delete data.

#### B. Search sentence segmentation

Use Jieba, AINLP, HanLP, PkuSeg and other Chinese word segmentation tools to segment the query sentence, and then input the word segmentation results into the search engine for query, which can improve the accuracy

of query. Compared with the massive data on the Internet, the data of office documents is less, and the user query is often an uncertain search sentence, but they want to get the corresponding results. For example, users want to query "China International Logistics Festival", but because the expression is inaccurate, it is written as "China logistics Festival", so the query results cannot be obtained. Therefore, it is easier to find out the corresponding results by using the word segmentation tool to divide into "China / logistics / Festival /".

#### C. Improve whoosh search strategy

When whoosh searches for keywords, by default, multiple keywords must be matched and hit at the same time to find the corresponding documents. For example, query "learning / education / activity /", implement the AND([term("title", "learning"), term("title", "education"), term("title", "activity")]) strategy, and the three keywords must be met at the same time. This paper changes the keyword relationship of "and" into "or" relationship, that is, OR([term("title", "learning"), term("title", "education"), term("title", "activity")]) to meet the needs of fuzzy query of official document users, so as to improve the hit rate of query.

#### D. Improve whoosh scoring strategy

Whoosh search score is based on BM25F algorithm. The accuracy of the score is adjusted by two values: document length weight B and saturation K1 of TF. On this basis, the paper also takes into account the impact of the score of the title and key information on the search results, multiplies the score by the strengthening weight according to the importance of the title and key information of the office document in the document, and adds the original score of whoosh to recalculate the score result. The method is as follows: assume that whoosh's original score is score0; Score of participle appearing in the title: score\_title0, the weight is 10; score in primary title: score\_title1, the weight is 5; score in secondary title: score\_title2, the weight is 2; score in tertiary title: score\_title3, the weight is 1; The scores in the key information are respectively score\_k1, score\_k2, ... score\_kn, the weight is 5. Then, the new score value score1 = score0 + 10 \* score\_title0 + 5 \* score\_title1 + 2 \* score\_title2 + score\_title3 + 5 \* (score\_k1 + score\_k2 + ... + score\_kn). The new scoring strategy, based on the original, takes into account the importance of the title and key information, making the calculation of scoring results more reasonable.

#### E. Calculate the relative probability of backward lookup

Whoosh search results are sorted according to the score value. The higher the score value, the higher the document ranking, but there is no score comparison between the results. Calculate the relative probability of backward search, which is convenient for users to quickly see the score difference of results and help users find results. The relative probability of backward search is based on the value with the highest score in the search results, which is recorded as highest\_score, and the score of each subsequent search result is recorded as next\_score, which can calculate the relative probability of backward search P:  $P = \text{next\_score} / \text{highest\_score} * 100\%$ . The relative probability of each backward search is displayed in the

front end in the form of pie chart, which provides a reference for users to analyze the search result document in the search result list, Examples of system operation results are as follows:



From left to right, the backward search relative probabilities of a certain search are given, which are 100, 67, 66, 66 and 39 respectively. Among them, the first search result is the benchmark probability, and the value is generally 100, and the rest decreases backward in order to assist user analysis and save search analysis time.

## VII. SYSTEM FRONT END DESIGN

It is mainly divided into retrieval result visualization, similar document recommendation, result preview confirmation, and retrieval result output. Among them, similar document recommendation and result preview confirmation are mainly used to assist users in finding and analyzing search results, as shown in Figure 3.

### A. Visualization of search results

First, filter the query results according to the user permissions, and only display and output the results for which the user has access permissions. Rewrite Haystack's highlight class MyHighLighter, set the highlight strategy, and adjust the length of the highlighted text. The information of each search result document is displayed in two lines. The first line is the title of the document, and the maximum character is 50; The second line is the full text search result display line. The display information is calculated from the first highlighted word segmentation retrieved, and 180 characters are displayed backward. The number of result documents displayed on each search page is 15, Examples of system operation results are as follows:

Title Title Title Title Title Title Title Title Test 5 [Download]

xxxx xxxx xxTest xxxx xxxx xxTest xxxx xxxx xxTest xxxx xxxx xxTest xxxx xxxx xxTest  
xxxx xxxx xxTest xxxx xxxx xxTest xxxx xxxx xxTest xxxx xxxx xxTest xxxx xxxx xxTest  
xxxx xxxx xxTest xxxx xxxx xxTest xxxx xxxx xxTest xxxx xxxx xxTest xxxx xxxx xxTest

Take a document retrieval "test" as an example. The first line of blue is the title, and the next line is the full-text text. After matching the "test" key, it is highlighted in the retrieval results at the same time, matching the text information from the inside of the document in depth, improving the retrieval accuracy.

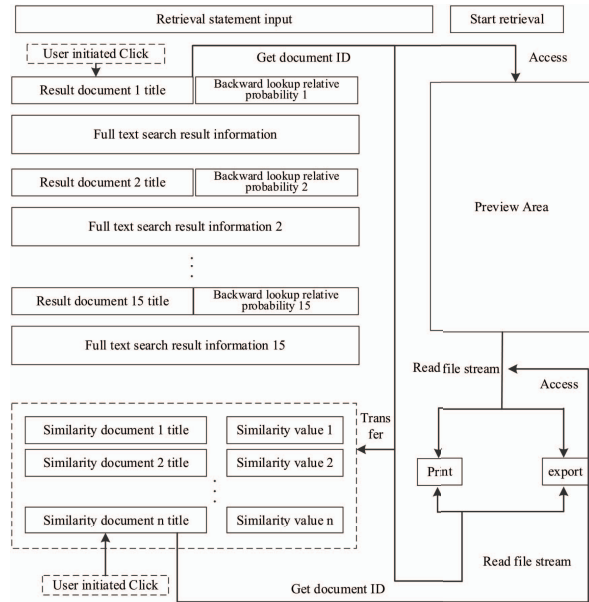


Figure 3. System front end design drawing

### B. Similar document recommendation

In the search results list, set a hyperlink for the title of each result document. When the user clicks to view a title, use the tag function to automatically capture the document ID and transfer it to the background. Find all similar document IDs and similar values in the document similarity statistics table. At the same time, in doc\_detail\_table find the document title, document storage address and preview file storage address, and return to the front page in the form of JSON string for preview and download output. Finally, the title of each similar document is displayed in the form of list on the front page, and the similar value is displayed in the form of horizontal histogram after each title, Examples of system operation results are as follows:

|                                                        |                   |        |
|--------------------------------------------------------|-------------------|--------|
| Title Title Title Title Title Title Title Title Test 5 | Similarity value: | 100%   |
| Title Title Title Title Title Title Title Title Test 3 | Similarity value: | 99.19% |
| Title Title Title Title Title Title Title Title Test 3 | Similarity value: | 99.19% |
| Title Title Title Title Title Title Title Title Test 2 | Similarity value: | 61.46% |
| Title Title Title Title Title Title Title Title Test 0 | Similarity value: | 61.02% |
| army of scientific research devoted to its mission     | Similarity value: | 29.78% |

After searching and clicking to preview a document, the system displays a list of 6 similar documents from top to bottom in the similar document recommendation area, and gives the corresponding similarity values, which are 100%, 99.19%, 99.19%, 61.46%, 61.02%, 29.78% from high to low, respectively, to assist users in quickly analyzing the corresponding documents they may be looking for.

### C. Result preview confirmation

In the search result list, set a hyperlink for each result. When the user clicks to view a result title, use the tag function to automatically capture the document ID, pass the value to the background, and query the corresponding preview file storage address in the doc\_detail\_table and return results to the front page again. According to the storage address of the preview file, the front page retrieves the web page preview file of the document and displays it in real time to help users confirm the query results. The file

is presented in the form of HTML page, and can also provide users with real-time text acquisition function to help users quickly download document information.

#### D. Search result output

Set the print and export buttons on the search page. When the user clicks the title link of the search result, the tag function will automatically capture the document ID and transfer the value to the background. The background according to the document ID finds the corresponding document storage address in the doc\_detail\_table and returned results to the foreground. When the user clicks the print and export button, the file stream can be read according to the address, printed or downloaded and exported to the user's local computer.

### VIII. SYSTEM FRONT END DESIGN

Starting from the practical requirements of multi-machine retrieval and combined with NLP technology, this paper puts forward a system design method of multi-machine retrieval, and expounds the design of the overall process, database structure and each module of the system. On the one hand, the implementation of the system can help users retrieve the result document quickly and accurately, and overcome the shortcomings of file name retrieval and stand-alone on-the-spot retrieval. On the other hand, by setting the document use permission, users can safely store the documents in the system database, facilitate document management, avoid the problem that the documents are easy to be lost when stored in the user terminal for a long time, and enable the documents to be shared according to the permission within a unit or business system, so as to improve the document circulation and use efficiency.

#### REFERENCES

- [1] Zhao Jingsheng, song Mengxue, Gao Xiang Overview of the development and application of natural language processing [J] Information technology and informatization, 2019 (07): 142-145
- [2] Neto J L , Freitas A A , Kaestner C A A . Automatic Text Summarization Using a Machine Learning Approach[C]// Advances in Artificial Intelligence, 16th Brazilian Symposium on Artificial Intelligence, SBIA 2002, Porto de Galinhas/Recife, Brazil, November 11-14, 2002, Proceedings. Springer Berlin Heidelberg, 2002.
- [3] Li Xianyu, Wu Xianfan A review of pre training technology for natural language processing [J] Computer science, 2020,47 (03): 162-173
- [4] Zhang Yunting, Ye Lin, Fang Binxing, et al Similar case retrieval algorithm based on word frequency inverse document frequency and Legal Ontology [J] Intelligent computer and application, 2021 (5): 229-235
- [5] Wang Yang, Wang Feifan, Zhang Shuyi, et al Spam text filtering of social platform based on TF-IDF and improved BP neural network [J] Computer system application, 2019, 28 (3): 126-132
- [6] Jiang Yuncheng, Yang Minxuan Qu Rong. Semantic similarity measures for formal concept analysis using linked data and WordNet [J]. Cations, 2019, 78 (14): 19807-19837.
- [7] biralatei Fawei, Jeff Z. Pan, Martin kollingbaum Adam Z. Wyner. A semi automated ontology construction for legal question answering [J]. New generation computing, 2019 37 (4): 43-478.
- [8] Zhao Fengpeng, Deng Xiang, Deng Hui, etc Research on Chinese text similarity calculation based on sequence alignment algorithm [J] Library and information work, 2021,11 (65): 100-111
- [9] Square ice Operational intention analysis method based on cosine similarity [a] China command and control society Proceedings of the 9th China command and Control Conference [C] China command and Control Society: China command and control society, 2021:4
- [10] Zhou Hai, Kong Yunbo, Kou Cheng, et al Whoosh based spatial data full-text retrieval method [J] Geospatial information, 2017 (11): 9 + 29-31
- [11] Matt Chaput. Whoosh Documentation: Release 2.7.4 [EB/OL]. (2016-06-19)[2016-07-12]. <http://media.readthedocs.org/pdf/whoosh/latest/whoosh.pdf>
- [12] Chen Yongjun, Xia Yanfeng, Gao Yuhang, et al Analysis and application of police text data based on NLP technology [J] Police Technology (2): 4
- [13] Tang Zhoulin Design and implementation of text retrieval system based on deep learning [D] Beijing University of Posts and telecommunications
- [14] GONG C, TANG J, ZHOU S, et al. Chinese Named Entity Recognition with Bert[C]. 2019 International Conference on Computer Intelligent Systems and Network Re-mote Control (CISNRC), Shanghai, 29-30 December 2019, 8-15.
- [15] LAN Z, CHEN M, Goodman S, et al. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations[J]. two thousand and nineteen
- [16] Shi Fenggui Implementation of Chinese text corpus preprocessing module based on Jieba Chinese word segmentation [J] Computer knowledge and technology, 2020, v.16 (14): 254-257 + 263
- [17] Shi Fenggui Research on Chinese text word segmentation and its visualization technology [J] Modern computer, 2020 (12): 131-138 + 148
- [18] Stuttering Chinese word segmentation [EB / OL] <https://github.com/fxsjy/jieba>.
- [19] Joulin A , Grave E , Bojanowski P , et al. Bag of Tricks for Efficient Text Classification[C]// Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers. two thousand and seventeen
- [20] Feng Zhiwei Word vector and its application in natural language processing [J] Foreign language audio visual teaching, 2019, No. 185 (01): 5-13
- [21] Zuo Weigang Research on campus network search engine based on Python [J] Electronic technology and software engineering, 2018, 144 (22): 40-42
- [22] Zhuang Xufei, Tian Xue Research and implementation of campus network search engine based on scrapy and elasticsearch [J] Science and technology information, 2019, 017 (029): 12-15
- [23] Huang Jizhou, Ding Shiqiang, Wang Haifeng, Liu Ting. Learning to recommend related entities with serendipity for web search users. ACM Transactions on Asian and Low-Resource Language Information Processing, 2018, 17(3): 25:1--25:22.
- [24] Chen D, Yi C, Brownlow B N, et al. Real-Time or Near Real-Time Persisting Daily Healthcare Data Into HDFS and ElasticSearch Index Inside a Big Data Platform[J]. IEEE Transactions on Industrial Informatics, 2017, 13(2):595-606.